JEEWAN

Join Educate Empower War Against Narcotics

Final Tech Stack + 3-Day Prototype Action Plan

1. Microservices Architecture

JEEWAN is built as a microservices application — each functional domain runs in its own Docker container with independent deployment, testing, and scaling. Nginx Ingress acts as the single API Gateway routing requests by path prefix.

1.1 Service Registry

Microservice	Technology	Port	Path Prefix	Database
Auth MS	FastAPI + Supabase Auth	:8001	/api/auth	PostgreSQL (Supabase)
SOS / Crisis MS	FastAPI + TextBee / Fonoster	:8002	/api/sos	PostgreSQL
Chatbot / AI MS	FastAPI + Ollama + Rasa	:8003	/api/chat	Redis (session)
Risk Assessment MS	FastAPI + HuggingFace	:8006	/api/risk	PostgreSQL
Gamification MS	FastAPI + Redis	:8004	/api/game	Redis + PostgreSQL
Maps / Rehab MS	FastAPI + OSM + Leaflet	:8005	/api/maps	PostgreSQL
AR Simulation MS	Next.js + MediaPipe	:3001	/ar	Stateless
Admin / Reporting MS	FastAPI + PostgreSQL	:8007	/api/admin	PostgreSQL
Frontend + API Gateway	Next.js 15 + Nginx Ingress	:3000	/	N/A

> *NOTE: Each MS has its own Dockerfile, requirements.txt / package.json, Kubernetes Deployment YAML, and Service YAML. Inter-service calls use Kubernetes DNS: http://auth-ms:8001.*

2. Final Tech Stack (100% Free)

2.1 Frontend

Tool / Library	Purpose	Why Chosen	Cost
Next.js 15 (React)	Main frontend framework	SSR, ISR static export, PWA support, best SEO	Free
Tailwind CSS (purged)	Styling	Purged bundle <10KB, works at 2G speeds	Free
shadcn/ui	UI components	Accessible, pre-built calm UI (blues/greens)	Free
Workbox (PWA)	Service worker + offline	Caches SOS + quiz offline, works with no network	Free
Next.js dynamic()	Lazy loading	AR/MediaPipe only loads when user taps it	Free
next-intl	i18n (English/Telugu/ Hindi)	NFR-09 multilingual requirement	Free
next/image + WebP	Image optimisation	Auto-compresses, <30KB per image at mobile	Free
Leaflet.js + OSM	Interactive maps	100% free, replaces Google Maps API	Free
Socket.io-client	Real-time chatbot + SOS	WebSocket connection to Chatbot MS	Free
MediaPipe (lazy)	AR face mesh	Drug-abuse aging simulation, loaded on demand	Free
Three.js (lazy)	AR overlay rendering	Renders MediaPipe face overlay effects	Free

2.2 Backend (Per Microservice)

Tool / Library	Purpose	Why Chosen	Cost
Python 3.12 + FastAPI	All backend microservices	Async, fast, auto OpenAPI docs, AI-native	Free
Pydantic v2	Data validation	Built into FastAPI, validates all API payloads	Free
SQLAlchemy 2.0	ORM	Type-safe DB access for PostgreSQL across MS	Free
LangChain (Python)	AI orchestration	DAST-10 scoring pipeline, chatbot memory, RAG	Free
Ollama + Llama 4 / Mistral	Self-hosted LLM chatbot	Zero API cost, replaces OpenAI API entirely	Free
Rasa Open Source	NLU chatbot framework	Empathy-trained intents for addiction counselling	Free
Hugging Face Inference	Risk classification model	DAST-10 scoring fallback, free inference endpoint	Free

Tool / Library	Purpose	Why Chosen	Cost
PyNaCl / cryptography	AES-256 + E2E encryption	Encrypts all counsellor chat sessions	Free
Socket.io (python-socketio)	WebSocket server	Real-time chatbot + SOS broadcasting	Free
Alembic	DB migrations	Version-controlled schema changes per MS	Free

2.3 Databases

Database	Used For	Hosting	Cost
PostgreSQL (Supabase)	Auth, counsellor profiles, RBAC, streaks, badges	Supabase free tier (500MB)	Free
MongoDB Atlas	Anonymous tip-off reports, no IP logging	Atlas free tier (512MB)	Free
Redis (Upstash)	Leaderboards, pledge counters, chatbot sessions	Upstash free tier (10K cmd/day)	Free

2.4 CI/CD Pipeline (Mandatory Stack)

Tool	Role	How Used	Cost
GitHub	Source control + webhook	Monorepo at /services/*, webhook fires Jenkins on push	Free
Jenkins (Docker)	CI orchestrator	Runs Jenkinsfile: Build > Scan > Test > Push > Deploy	Free
SonarQube Community	Code quality gate	Blocks deploy if coverage drops or vulnerabilities found	Free
Docker + Docker Hub	Containerisation + registry	One image per MS, pushed to Docker Hub free tier	Free
Kubernetes (Minikube)	Container orchestration	Local K8s cluster, Deployment + Service YAML per MS	Free
Nginx Ingress	API Gateway	Routes /api/auth → auth-ms, /api/sos → sos-ms, etc.	Free
Prometheus + Grafana	Monitoring + alerting	SOS latency <3s SLA, uptime 99.9% dashboard	Free

2.5 Testing Stack (Full, Free)

Layer	Tool	Equivalent To	Used For
Python unit + integration	Pytest 8 + pytest-asyncio	JUnit5 (Java)	All FastAPI route tests across all MS

Layer	Tool	Equivalent To	Used For
Python coverage	pytest-cov (lcov/XML)	JaCoCo (Java)	Coverage reports piped to SonarQube
JS component tests	Jest + React Testing Library	JUnit5 (Java)	Next.js component tests, SOS button, quiz UI
End-to-end browser tests	Selenium 4 + pytest	Selenium (standard)	Full user flows: SOS, guest mode, quiz, chatbot
E2E secondary (headless)	Playwright	Cypress	Smoke tests in Jenkins pipeline (faster)
API contract tests	Schemathesis	REST Assured (Java)	Auto-generates tests from FastAPI OpenAPI schema
Load / performance	Locust	Gatling (Java)	SOS endpoint: validates <3s at 500+ concurrent users
Security scan (Python)	Bandit	SpotBugs (Java)	Detects hardcoded secrets, SQL injection in FastAPI
Security scan (static)	SonarQube Community	PMD / Checkstyle	Code smells, duplications, vulnerability rules
AI chatbot NLU tests	rasa test	Custom NLP harness	Validates intent classification and dialogue flows
Async mock	pytest-httpx + fakeredis	Mockito (Java)	Mocks external calls (OSM, TextBee) in unit tests

> *NOTE: Selenium is used for full E2E user flows that require real browser rendering (SOS panic button click, AR camera permission dialog, WebSocket chatbot flows). Playwright is used in CI pipeline as it runs headless faster. Both are free and open-source.*

---

3. 3-Day Prototype Action Plan

Target: Working prototype by Monday morning. Three developers recommended — one frontend, one backend, one DevOps/infra. Tasks below are prioritised by dependency order.

Day 1 — Friday: Infrastructure + Auth + SOS

Morning (9AM–1PM): Repo + Docker + CI skeleton

1. Create GitHub monorepo: jeewan/ with /services/auth, /services/sos, /services/chatbot, /services/gamification, /services/maps, /services/risk, /services/ar, /services/admin, /frontend
2. Write Dockerfile for each service (FastAPI: python:3.12-slim base; Next.js: node:20-alpine base)
3. Write docker-compose.yml spinning all 9 services + PostgreSQL + MongoDB + Redis on shared network
4. Run Jenkins locally: docker run -p 8080:8080 jenkins/jenkins:lts

5. Run SonarQube locally: docker run -p 9000:9000 sonarqube:community
6. Configure Jenkins job: GitHub webhook → Jenkinsfile with stages: Build, SonarQube, Test, Docker push
7. Write Kubernetes Deployment + Service YAML for Auth MS and SOS MS (remaining done Day 2–3)
8. Install Minikube + kubectl, start cluster, enable Nginx Ingress addon

Afternoon (2PM–7PM): Auth MS + SOS MS

9. Auth MS (FastAPI): Implement /register (OTP), /login, /guest-token, /me endpoints with Supabase Auth JWT
10. Auth MS: Role middleware — Guest, User, Counsellor, Volunteer, Admin — using FastAPI dependencies
11. SOS MS (FastAPI): Implement /sos/trigger endpoint capturing GPS coordinates from request body
12. SOS MS: Integrate TextBee REST API — POST to TextBee to send SMS with Google Maps link to emergency contacts
13. SOS MS: Target <3 second response from button press to SMS sent — test with Locust from Day 1
14. Frontend (Next.js): Create SOS panic button page — accessible in 1 click from home screen (NFR-07)
15. Frontend: Wire SOS button to SOS MS via fetch, show confirmation toast on success
16. Write Pytest tests for Auth MS (register, login, guest, RBAC) and SOS MS (trigger, latency)

Day 2 — Saturday: Core Features + CI Pipeline

Morning (9AM–1PM): Chatbot + Risk Assessment MS

17. Chatbot MS: Install Ollama, pull Llama 3.2 or Mistral 7B model, expose FastAPI /chat endpoint
18. Chatbot MS: Implement Rasa NLU — create intents: withdrawal_symptoms, addiction_signs, legal_rights, crisis_support
19. Chatbot MS: Connect Socket.io for real-time streaming response to frontend
20. Risk MS: Implement DAST-10 quiz — 10 questions scored 0-4, POST /risk/assess endpoint
21. Risk MS: If score > 6 (high risk threshold), auto-flag to admin dashboard and return get_help modal payload
22. Risk MS: Wire Hugging Face inference API as fallback classifier for risk scoring
23. Frontend: Build chat UI page (calming blue/green tones, NFR-08) connected via Socket.io
24. Frontend: Build DAST-10 quiz UI — one question per step, progress bar, result screen

Afternoon (2PM–7PM): Maps + Gamification + AR

25. Maps MS: Integrate OpenStreetMap + Leaflet.js — /maps/nearby endpoint accepts lat/lon, returns rehab centres within 50km
26. Maps MS: Nominatim reverse geocoding for address lookup, filter by government/private type
27. Gamification MS: /pledge/daily endpoint — one click per 24 hours, streak counter stored in Redis sorted set
28. Gamification MS: Badge awards — Awareness Champion (3 perfect quizzes), Drug-Free Ambassador (30-day streak)

29. Gamification MS: /leaderboard/institutions endpoint — Redis sorted set ranking institutions by participation points
30. AR MS (Next.js page): Import MediaPipe via Next.js dynamic() — only loads on user tap
31. AR MS: Use MediaPipe FaceMesh to detect 468 face landmarks, apply aging/sores canvas overlay
32. CI pipeline: Configure Jenkins to run pytest for all Python MS and jest for frontend on every GitHub push

Day 3 — Sunday: Polish + Testing + K8s Deploy

Morning (9AM–1PM): Anonymous Reporting + Admin + Integration

33. Anonymous Reporting: POST /tips/submit storing to MongoDB with no IP logging (NFR-01), strip all metadata
34. Admin MS: Dashboard endpoints — GET /admin/cases (flagged risk cases), GET /admin/stats (addiction trends)
35. Counsellor Module: GET /counsellors (profiles), POST /appointments/book (video/audio/chat slots)
36. Survivor Stories: GET /stories endpoint returning curated video testimonial feed
37. Nginx Ingress: Configure path-based routing to all 9 MS in Kubernetes cluster
38. K8s: Apply all remaining Deployment + Service YAMLs, verify all pods running (kubectl get pods)
39. Prometheus: Deploy in K8s, configure scrape targets for /metrics endpoint on each MS
40. Grafana: Import dashboard — SOS p95 latency gauge, chatbot uptime panel, active users counter

Afternoon (2PM–6PM): Testing + Low-Bandwidth + Demo

41. Selenium E2E tests: Write test suite for SOS flow, guest mode, DAST-10 quiz, leaderboard, chatbot
42. Playwright smoke tests: Add to Jenkinsfile as final pipeline stage (headless, faster for CI)
43. Locust load test: Simulate 500 concurrent users hitting SOS endpoint, confirm p95 < 3 seconds
44. Schemathesis: Run against all FastAPI services to auto-test OpenAPI contract
45. SonarQube: Fix any critical issues flagged — aim for A rating on reliability, security, maintainability
46. PWA: Test offline mode — disconnect network, verify SOS screen and quiz still load from service worker cache
47. i18n: Add Telugu and Hindi strings for SOS, home, and quiz screens using next-intl
48. Final demo run: Walk through complete user journey — guest SOS, quiz risk flag, chatbot, AR, leaderboard
49. Write README.md with architecture diagram, setup instructions, and docker-compose up command

---

4. Selenium Integration — Step-by-Step

Selenium 4 is used for full end-to-end browser tests that require real browser rendering — particularly important for the SOS panic button (must verify 1-click access from launch), WebSocket chatbot, and AR camera permission. Playwright handles headless CI smoke tests.

4.1 Installation

50. Install Selenium and browser drivers in the test environment:

```
pip install selenium==4.x pytest-selenium webdriver-manager
```

51. Selenium 4 includes Selenium Manager — it auto-downloads ChromeDriver matching your Chrome version. No manual driver setup needed.
52. Install Chrome/Chromium in your Docker test container:

```
RUN apt-get install -y chromium chromium-driver
```

4.2 Configuration (conftest.py)

Create a shared conftest.py in your /tests/e2e/ folder:

```
import pytestfrom selenium import webdriverfrom selenium.webdriver.chrome.options import
Options@pytest.fixture(scope='session')def driver():    opts = Options()
opts.add_argument('--headless=new')    opts.add_argument('--no-sandbox')
opts.add_argument('--disable-dev-shm-usage')    d = webdriver.Chrome(options=opts)
d.implicitly_wait(10)    yield d    d.quit()
```

4.3 Writing Tests

Example: SOS button 1-click access test (NFR-07)

```
from selenium.webdriver.common.by import Byfrom selenium.webdriver.support.ui import
WebDriverWaitfrom selenium.webdriver.support import expected_conditions as ECdef
test_sos_button_accessible_one_click(driver):    driver.get('http://localhost:3000')
sos_btn = WebDriverWait(driver, 5).until(        EC.element_to_be_clickable((By.ID,
'sos-panic-button'))    )    assert sos_btn.is_displayed(), 'SOS button must be visible
on home screen'    sos_btn.click()    confirm = WebDriverWait(driver, 5).until(
EC.visibility_of_element_located((By.ID, 'sos-confirmation'))    )    assert
confirm.is_displayed()
```

4.4 Jenkins Integration

53. Add a Selenium stage to your Jenkinsfile after the unit test stage:

```
stage('Selenium E2E Tests') {    steps {        sh 'docker-compose up -d'  // Start all
services        sh 'sleep 15'               // Wait for services        sh 'pytest
tests/e2e/ -v --junitxml=reports/selenium.xml'    }    post {        always {
junit 'reports/selenium.xml'        }    }}
```

54. Selenium runs in the same Docker network as the Next.js frontend, so it connects to http://localhost:3000 (or the container name in compose).
55. JUnit XML report is picked up by Jenkins and shown in the test results panel.

4.5 Test Scenarios to Cover

Test Scenario	Selenium Test File	Requirement Covered
SOS button visible + clickable on home (1 click)	test_sos.py	NFR-07 (1-click access)
Guest user can access SOS without login	test_guest.py	FR-01 (anonymous access)

Test Scenario	Selenium Test File	Requirement Covered
DAST-10 quiz completes and shows result	test_quiz.py	FR-08 (risk assessment)
Chatbot responds within 5 seconds	test_chatbot.py	FR-07 (24/7 chatbot)
Leaderboard loads and shows institutions	test_leaderboard.py	FR-12 (leaderboards)
Anonymous tip-off form submits successfully	test_tipoff.py	FR-05 (anonymous reporting)
Rehab centre map loads with markers	test_maps.py	FR-06 (rehab locator)
Pledge button increments streak	test_pledge.py	FR-10 (daily pledge)
Language toggle switches to Telugu/Hindi	test_i18n.py	NFR-09 (language support)

> *NOTE: For the AR module Selenium test: use driver.execute_script() to mock getUserMedia (camera API) so the test does not require a physical camera in CI.*

5. Low-Bandwidth Strategy Summary

JEEWAN must work on 2G networks in remote areas (NFR-11). The following optimisations are non-negotiable:

- Static export critical pages (education, quiz, articles) — served as plain HTML, zero JavaScript required
- Tailwind CSS purged at build — final CSS bundle under 10KB
- AR (MediaPipe ~2MB) loaded only via Next.js dynamic() when user explicitly taps the AR screen
- PWA service worker (Workbox) caches SOS screen, quiz, and emergency contacts offline
- All images served as WebP via next/image with blur placeholder — no image exceeds 30KB on mobile
- Font subset: system font stack fallback; if custom font used, subset to Telugu + Devanagari + Latin scripts only (~12KB)
- Survivor story videos lazy-loaded — thumbnail only loads on scroll, video only plays on tap
- API responses gzip-compressed by Nginx Ingress — reduces payload by 60-70%
- Text-based chatbot and quiz function on 2G as they send only JSON, no media